



Experiment No.: 4

1. Title: Use function types, scope, and closures; apply try-catch; build a palindrome checker.

2. Software/Tools Required: Visual Studio Code (VS Code) Google Chrome/web browser, HTML, JavaScript.

3. Theory:

Function Types: Demonstrates anonymous functions (assigned directly to variables without a name) and arrow functions (concise ES6 => syntax).

Scope: Differentiates between **global variables** (accessible throughout the script) and **local variables** (restricted to the block or function where they are declared).

Closures: Occurs when an inner function retains access to its parent function's scope, preserving state variables (like `count`) even after the outer function finishes executing.

Error Handling: Uses `try...catch` and `throw` statements to validate input types and gracefully handle runtime exceptions without crashing the program.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, NAGPUR
Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A++' Grade | Awarded Category - I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Palindrome Algorithm: A string is split into an array of characters, reversed, joined back into a string, and compared against the original input using strict equality (===).

4.code:

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Advanced Palindrome Checker</title>

    <style>

        body {

            font-family: Arial, sans-serif;

            margin: 40px;

            background-color: #f4f4f9;

        }

        .container {

            max-width: 500px;

            background: white;
```

```
padding: 30px;

border-radius: 8px;

box-shadow: 0 4px 15px rgba(0,0,0,0.1);

}

input, button {

width: 100%;

padding: 12px;

margin: 10px 0;

border-radius: 4px;

border: 1px solid #ccc;

box-sizing: border-box;

font-size: 16px;

}

button {

background-color: #007bff;

color: white;

border: none;

cursor: pointer;

font-weight: bold;

}

button:hover {

background-color: #0056b3;
```

```
    }

    .result {

        margin-top: 20px;

        padding: 15px;

        border-radius: 4px;

        font-weight: bold;

    }

    .success { background-color: #d4edda; color: #155724; }

    .error { background-color: #f8d7da; color: #721c24; }

    .stats { font-size: 14px; color: #555; margin-top: 10px; }

</style>

</head>

<body>

<div class="container">

    <h2>Palindrome Checker Experiment</h2>

    <p>Enter text to verify if it is a palindrome. Empty inputs or numbers
will trigger validation errors.</p>

    <input type="text" id="userInput" placeholder="Type a word or
phrase...">

    <button id="checkBtn">Check String</button>
```

```
<div id="outputDisplay"></div>

</div>

<script>

  // 1. SCOPE: Global Variable

  const globalSystemName = "Palindrome Engine v1.0";

  // 2. CLOSURE: Tracks total checks processed via persistent inner
state

  const createTracker = () => {

    let count = 0; // Local variable preserved by the closure

    return () => {

      count++;

      return count;

    };

  };

  const getCheckCount = createTracker();

  // 3. FUNCTION TYPES: Anonymous function bound to click event listener

  document.getElementById('checkBtn').addEventListener('click',
function() {

    // SCOPE: Block-scoped local variables

    const inputField = document.getElementById('userInput');
```

```
const outputDiv = document.getElementById('outputDisplay');

const rawInput = inputField.value;

// 4. ERROR HANDLING: Try...Catch structure

try {

    // Validation rules

    if (!rawInput.trim()) {

        throw new Error("Input cannot be empty. Please enter a
word.");

    }

    if (!isNaN(rawInput) && rawInput.trim() !== "") {

        throw new Error("Numbers are not valid words. Please enter
text.");

    }

    // 5. FUNCTION TYPES: Arrow function for palindrome evaluation

    // PALINDROME ALGORITHM: split -> reverse -> join -> strict
equality

    const isPalindrome = (str) => {

        const cleanStr = str.toLowerCase().replace(/^[a-z0-9]/g,
');

        const reversedStr = cleanStr.split('').reverse().join('');

        return cleanStr === reversedStr;

    };

}
```

```
const result = isPalindrome(rawInput);

const totalChecks = getCheckCount(); // Invoking the closure

// Render matching output UI

if (result) {

    outputDiv.className = "result success";

    outputDiv.innerHTML = `"${rawInput}"</strong> is a
palindrome!<br>`;

} else {

    outputDiv.className = "result error";

    outputDiv.innerHTML = `"${rawInput}"</strong> is
NOT a palindrome.<br>`;

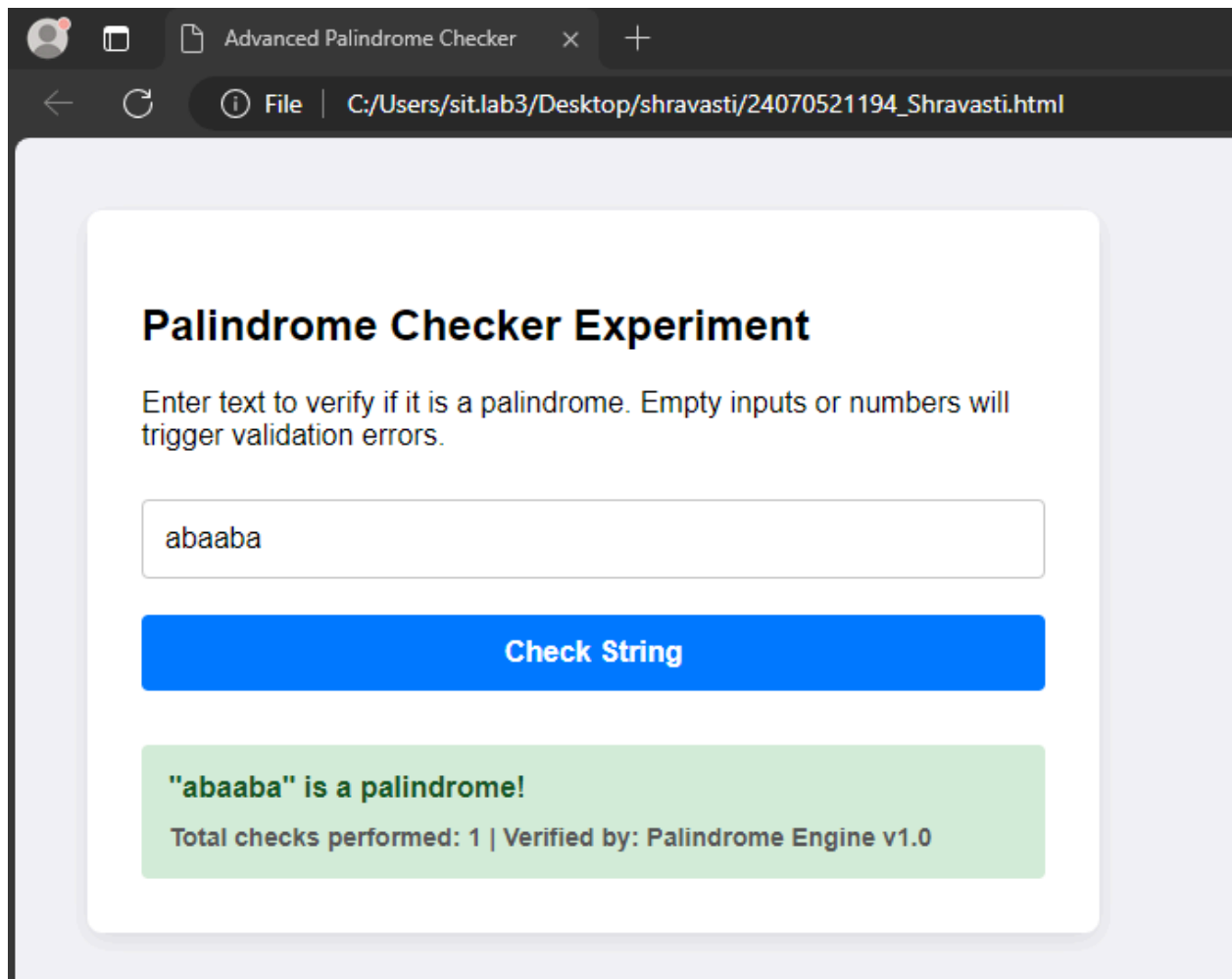
}

// Append closure statistics and global variable string

outputDiv.innerHTML += `
```

```
${error.message}`;  
  
    }  
  
    });  
  
</script>  
  
</body>  
  
</html>
```

5 Output:



5. Result/Conclusion: In this experiment, fundamental

JavaScript concepts—including anonymous/arrow functions, scope rules, closures, and `try...catch` error handling—were successfully implemented. Using these techniques alongside array manipulation methods (`split`, `reverse`, `join`), a functional and error-resistant palindrome checker was constructed.